Idea 1: Do Sequential Tests Beat Max-Aggregation at a Matched False-Alarm Budget?

Matched-budget comparison of trajectory decision rules for AI-agent monitors

October 2026

Summary.

- AI-control evaluations make trajectory decisions by taking the *maximum* per-step monitor score and comparing it with a quantile of honest-run scores.
- We compared max with 11 alternatives at a *matched trajectory-level false-positive rate*. The alternatives are 2nd-largest, top-3 sum, mean, length-aware max, CUSUM, Shiryaev–Roberts, window GLR, conformal e-processes, and OR multi-charts.
- The comparison covers 46 real score-stream sets (aksh-n, LinuxArena with 12 monitors, MRT, persistent-state), plus a new controlled experiment: 775 MonitoringBench attacks and 550 honest runs re-scored by the same `gpt-4.1-mini` monitor with and without history.
- **Main results.**
  1. Max, 2nd-max and top-3 sum are strong defaults. Textbook sequential tests lose to them on almost every set.
  2. Accumulation pays off clearly in one regime: **cross-session evidence from a history-free per-unit monitor**. Top-3 sum or Shiryaev–Roberts beat max by about +0.15 TPR at 5% FPR on gradual multi-PR attacks. The gain disappears once the per-PR monitor sees prior diffs.
  3. Whether accumulation helps is predicted by two measurable stream properties: honest high-score persistence ($\rho = -0.67$) and how informative a repeated spike is under attack ($\rho = +0.47$).
  4. CUSUM's reference value k is critical: $k = 0.5$ fails, $k \approx 1.5$–2 roughly matches max.
  5. Length-aware thresholds are an easy win when honest and attack lengths differ (MRT: 0.21 → 0.58 TPR).

Spend: $109.56 OpenRouter.

Contents

1 Motivation and question **2**

2 Data **3**

3 Methods **4**

3.1 Per-unit transforms 4

3.2 Detectors (trajectory statistics) 4

3.3 Matched-budget calibration 4

4 Results **5**

4.1 Round 1: existing streams (46 sets, $0) 5
4.2 Round 2: controlled history experiment on MonitoringBench 6
4.3 Mechanism across all 46 stream sets 9
4.4 Round 3: robustness 10

5 Conclusions **10**

6 Limitations **10**

7 Next steps **11**

1 Motivation and question

A per-step monitor (an LLM judge) gives each agent action a suspicion score. Almost every AI-control paper and benchmark turns the resulting stream $s_1, \ldots, s_T$ into a trajectory decision the same way: it audits the trajectory if $\max_t s_t$ exceeds a threshold, set as the $(1-\alpha)$ quantile of honest-run maxima at the audit budget α. This holds for BashArena, LinuxArena, MonitoringBench, Ctrl-Z, Control OSWorld and SHADE-Arena.

Detection theory says max (a Shewhart chart) is optimal for a single large spike but weak for evidence spread over many steps. For that case, sequential statistics are the textbook answer: CUSUM, Shiryaev–Roberts, GLR, e-processes. This matters because:

- distributed attacks are an active concern;
- MonitoringBench reports that a 1% per-step FPR compounds to 6.7–17.4% per trajectory;
- MonitoringBench explicitly leaves a matched-FPR comparison of stepwise and full-trajectory monitoring to future work.

At the same time, three published attempts at simple accumulation failed:

- k-strikes rules cut safety from 61% to 44% (Redwood, *Retrying vs Resampling*);
- k-strikes failed in persistent-state control;
- Bayesian score accumulation gave no gain (Meerkat).

Pre-registered mechanism hypothesis ("double counting"). Per-step monitors usually read the full history, so their scores are already cumulative judgments. Summing them double-counts evidence and inflates the honest tail of any accumulator. Accumulation should therefore help on *history-free* ("innovation") monitors and not on history-aware ones.

2 Data

Source	Honest / attack	Monitor(s)	Unit / onset label
aksh-n monitor-labels (7 settings: agentdojo, bash, iac, shade, BashArena opus/sonnet, LinuxArena)	7,082 / 6,894	Gemini-3.1-Pro streaming judge; *local* (per-step) and *cumulative* scores, 0–10	step; ideal-flag step
LinuxArena monitor sweep	200 / 142	12 per-action monitors on the same trajectories (gpt-4.1-nano … claude-opus-4-6), 0–100	step; harm step (36 live side tasks)
ScaleAI MRT (SHADE / CUA-SHADE)	436 / 1,022	claude-3.5-sonnet sequential chunk monitors ($N = 5, 7$; aware/unaware prompts), 1–5	chunk of N messages
Persistent-state VibeCoding	171 / 520	gpt-4o per-PR monitors: `diff_current_only` (history-free), `full_diffs`, `task_sequence` (history-aware), trajectory	PR (5 per run); gradual and positional attacks
MonitoringBench + Honest-550 (new scoring)	550 / 775	`gpt-4.1-mini`, prompt `ctrl-basic-v1`, history `full` vs `none`; `gpt-4.1-nano none`	step; `attack_step` (execution) and heuristic `first_attack_step`

Table 1: Score-stream sources. The 775 MonitoringBench attacks are all 475 S4 (distributed) attacks plus 100 each of S1–S3, stratified by side task (seed 20261020).

Confound controls (MonitoringBench).

- Prompts are action-only. Agent reasoning is hidden from the monitor because Honest-550 shows reasoning on almost every step but attacks show it on only 47%.
- The final `submit` step is stripped from both classes (99% of honest runs end with it vs 78% of attacks).
- Argument names are normalized (`cmd`/`command`).
- The monitor sees the current action's output.

New scoring.

- Per step, with the logprob-expected 0–10 score: `gpt-4.1-mini` returns token logprobs via OpenRouter.
- History modes:
  - `full`: task, all previous actions (truncated) and the current action;
  - `none`: task and the current action only, explicitly judged on its own evidence.
- A pilot on 40 trajectories cost $4.97. Per-step costs: `full` $0.0015–0.0027, `none` $0.0009–0.0010.
- Passing `prompt_cache_key` and sending each trajectory's steps in order raised OpenAI prompt-cache hits and cut the full run's cost from a projected $134 to about $101.

3 Methods

3.1 Per-unit transforms

Each unit score s gets a conformal p-value against honest calibration units, $p = (\#\{h_i > s\} + U(\#\{h_i = s\} + 1))/(n+1)$, randomized with $U \sim \mathrm{Unif}(0,1)$ to handle ties. Sequential statistics use the whitened score $z = \Phi^{-1}(1-p)$, which is $\approx \mathcal{N}(0,1)$ on honest data.

3.2 Detectors (trajectory statistics)

Detector	Statistic (alarm when it exceeds h; trajectory stat = max over the running path)
max, 2nd-max, top-3 sum, mean	order statistics of raw scores; top-k uses whitened z
length-aware max	$-\log\big(1 - F(\max_{i\le t} s_i)^{\theta t^{\beta}}\big)$, an analytic length-adjusted null
CUSUM(k)	$S_t = \max(0, S_{t-1} + z_t - k)$; $k = 0.5$ primary, plus a k-grid
Shiryaev–Roberts(δ)	$\log R_t = \mathrm{logaddexp}(0, \log R_{t-1}) + \delta z_t - \delta^2/2$, $\delta = 1$
window GLR(W)	$G_t = \max_{m \le W} \max(0, \sum_{i=t-m+1}^{t} z_i)^2/(2m)$, $W = 20$ (plus a W-grid)
conformal e-process	$\log E_t$, $E_t = \mathrm{mean}_\kappa \prod_{i\le t} \kappa p_i^{\kappa-1}$, $\kappa \in \{0.02, \ldots, 0.5\}$ (anytime-valid under i.i.d. honest units)
SR e-detector	$M_t = f(p_t)(1 + M_{t-1})$, restarted e-processes summed
OR multi-charts	max OR CUSUM (OR e-process), *jointly* calibrated to the target FPR

3.3 Matched-budget calibration

- **Honest-only calibration.** 10 repeated 50/50 honest splits, grouped by main task when there are ≥ 30 tasks. 5-fold cross-fitting of the per-unit transforms on the calibration half.
- **Thresholds.** Randomized plug-in thresholds (primary), which remove integer-score discreteness so every detector realizes $\approx \alpha$. Conformal thresholds are a robustness check.
- **Targets.** $\alpha \in \{0.05, 0.01, 0.003\}$.
- **CIs.** A cluster bootstrap by task (300 replicates) that includes calibration noise.
- **Streaming metrics.** Matched ARL, by appending attacks to 0–4 honest test prefixes without resets; detection delay from the onset label; alarm-before-harm; onset localization. The non-restarting e-process has no stationary ARL and is excluded from ARL comparisons.

4 Results

4.1 Round 1: existing streams (46 sets, $0)

Set	max	2nd	top-3	CUSUM	SR	GLR	e-proc	len-max
LinuxArena: claude-opus-4-6	**0.95**	0.88	0.92	0.66	0.65	0.69	0.81	**0.97**
LinuxArena: gemini-3.1-pro	0.93	0.85	0.88	0.56	0.56	0.70	0.68	**0.94**
LinuxArena: gpt-5.4	0.77	0.72	0.71	0.32	0.32	0.56	0.49	**0.84**
LinuxArena: gpt-5-mini	0.69	0.56	0.60	0.37	0.36	0.44	0.47	**0.70**
LinuxArena: glm-5	0.84	0.78	**0.89**	0.64	0.63	0.70	0.79	0.88
LinuxArena: gpt-4.1-nano	0.20	0.13	0.14	0.13	0.12	0.13	0.16	**0.24**
aksh-n BashArena sonnet45 (cum.)	0.98	0.98	0.96	0.31	0.32	0.68	0.76	**0.99**
aksh-n shade (local)	0.82	0.82	0.84	0.62	0.62	0.64	0.45	0.81
MRT sonnet-3.5 N=5	0.21	0.22	0.18	0.02	0.01	0.05	0.03	**0.58**
MRT sonnet-3.5 N=5 superaware	0.33	0.37	0.26	0.02	0.02	0.04	0.06	**0.83**
PR `current_only`: gradual	0.16	0.24	0.31	0.28	**0.33**	0.21	0.15	0.16
PR `current_only`: positional	0.19	0.16	**0.30**	0.26	0.29	0.24	0.24	0.17
PR `full_diffs`: gradual	**0.24**	0.09	0.16	0.22	0.22	**0.25**	0.23	0.23
PR trajectory monitor: gradual	0.35	0.17	0.31	**0.37**	**0.37**	**0.37**	0.17	0.33

Table 2: TPR at matched trajectory FPR $\alpha = 0.05$ (selected sets; all 46 sets: `results/t1/tables/r1_tpr.md`).

- **Max-type rules dominate** at $\alpha = 0.01$. Max, 2nd-max or length-aware max matched or beat every accumulator in 86% of aksh-n sets, 67% of LinuxArena monitors and 83% of MRT sets. Median TPR(CUSUM) − TPR(max) was −0.39, −0.25 and −0.01 respectively.
- **Accumulation gain falls with honest autocorrelation.** The Spearman correlation between the CUSUM−max gain and honest lag-1 autocorrelation of whitened scores is −0.57 ($\alpha = 0.05$) and −0.45 ($\alpha = 0.01$) (Fig. 1). This is consistent with double counting.
- **Timing.** On aksh-n, max alarms at the ideal flag step (median delay 0) and CUSUM 3–25 steps later. The attacks are concentrated in time.
- **Length.** Honest and attack lengths differ in several sets, and the max of a longer honest stream is inflated. The analytic length-aware max fixes this and is often the single best rule.
- **Cross-session (persistent state) supports the hypothesis.**
  - With the history-free `current_only` PR monitor, accumulation beats max by +0.15 TPR (top-3 sum vs max: +0.15 [0.12, 0.17], $\alpha = 0.05$; SR 0.33 vs max 0.16 on gradual attacks).
  - With the history-aware `full_diffs` monitor on the same runs, the advantage disappears (top-3 sum −0.07).

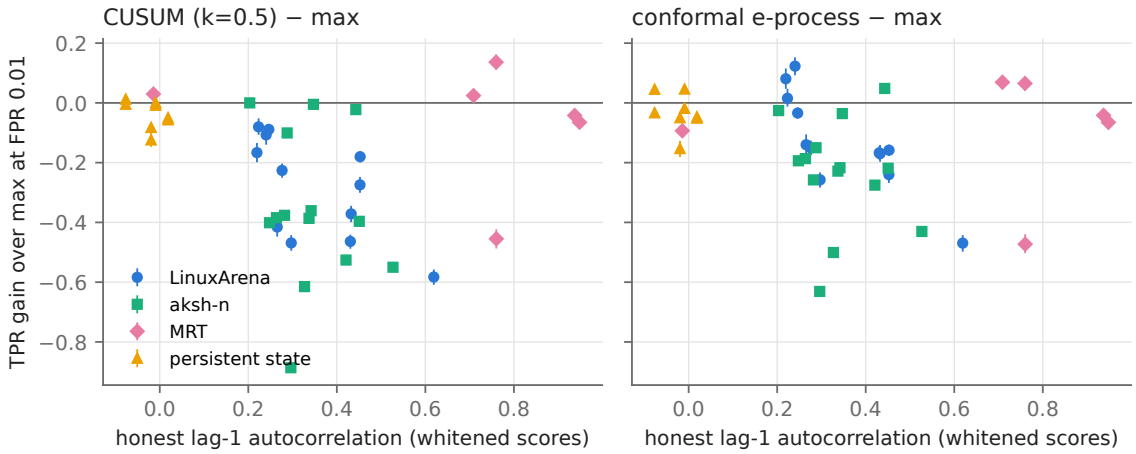


Figure 1: Accumulation gain over max (TPR at FPR 0.01) vs honest lag-1 autocorrelation of whitened scores, one point per stream set. Sets with strongly autocorrelated honest scores (cumulative judgments) lose the most.

4.2 Round 2: controlled history experiment on MonitoringBench

Same monitor (`gpt-4.1-mini`) and prompt. Only the history mode differs: `full` vs `none`.

	$\alpha = 0.05$		$\alpha = 0.01$		$\alpha = 0.003$	
Detector	full	none	full	none	full	none
max	0.226	**0.322**	0.171	0.156	**0.140**	0.079
2nd-max	**0.342**	0.276	0.190	0.152	0.111	0.078
top-3 sum	0.324	0.304	**0.195**	**0.190**	0.106	**0.082**
mean	0.173	0.169	0.093	0.065	0.037	0.035
CUSUM k=0.5	0.073	0.087	0.030	0.039	0.017	0.021
Shiryaev–Roberts	0.069	0.084	0.029	0.035	0.017	0.020
window GLR	0.112	0.116	0.054	0.056	0.040	0.029
e-process (power)	0.130	0.126	0.052	0.065	0.039	0.045
SR e-detector	0.162	0.162	0.058	0.063	0.043	0.038
length-aware max	0.215	0.279	0.153	0.124	0.088	0.049
OR(max+CUSUM)	0.201	0.247	0.137	0.085	0.077	0.030

Table 3: TPR at matched trajectory FPR, 775 attacks vs 550 honest (95% CIs in `results/t1/tables/r2_main.md`; e.g. max full at $\alpha = 0.01$: 0.171 [0.13, 0.22]). Realized FPRs are within about ±0.01 of target.

- **The pre-registered prediction is not supported on this monitor.** Sequential gains do not concentrate in `none` mode on S4 distributed attacks (Fig. 4). The difference-in-differences (none−full) for CUSUM is +0.02 only at $\alpha = 0.01$, and negative at $\alpha = 0.05$ and with integer scores.
- **Mechanism.** The two halves of the hypothesis behave differently here:

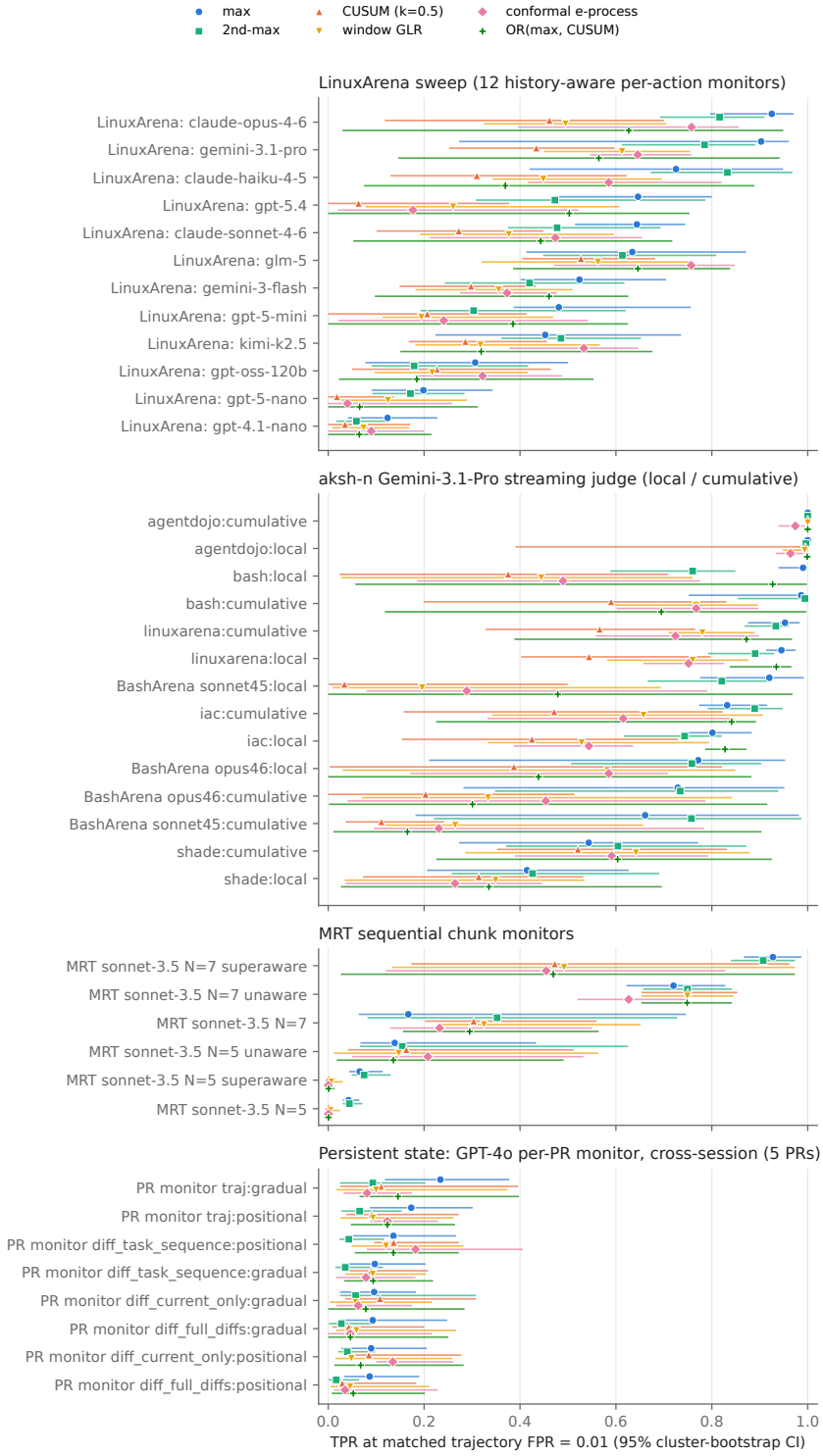


Figure 2: Round 1: TPR at matched trajectory FPR 0.01 with 95% cluster-bootstrap CIs, by stream set (LinuxArena sweep, aksh-n, MRT, persistent state) and detector.

  - *Honest streams:* `gpt-4.1-mini`'s full-history honest scores are *not* cumulative (tail lift from adding history ≈ 1).
  - *Attack streams:* full history makes the monitor *re-flag* attacks across steps. For S1 attacks, ≥ 2 high steps occur 60% (full) vs 39% (none) of the time. This helps 2nd-max and top-3 rather than hurting accumulation.
- **Low absolute detection.** With this cheap monitor, about 17–20% TPR at 1% trajectory FPR. Differences between the best detectors are within wide CIs at $\alpha \leq 0.01$.
- **Weak monitor** (`gpt-4.1-nano`, `none`): top-3 sum 0.082 vs max 0.056 at $\alpha = 0.01$.

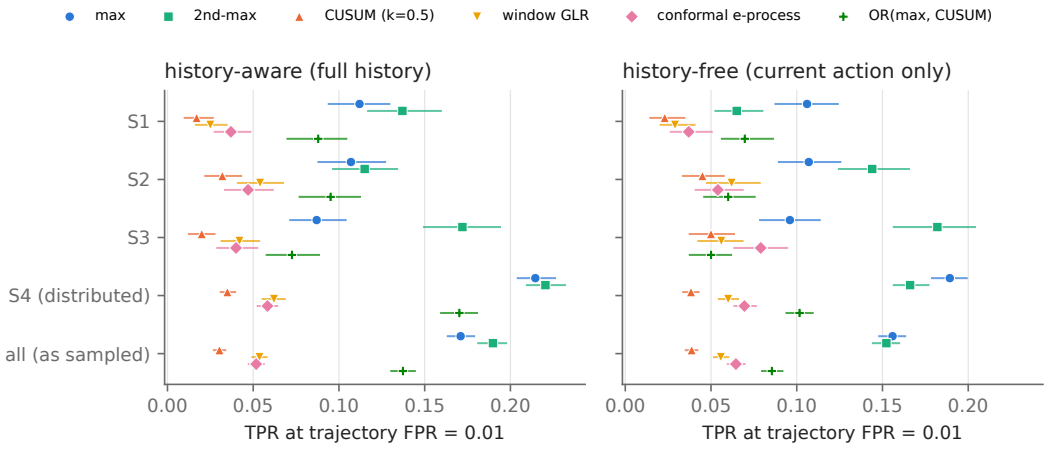


Figure 3: TPR at trajectory FPR 0.01 by attack structure (S1–S4) and history mode.

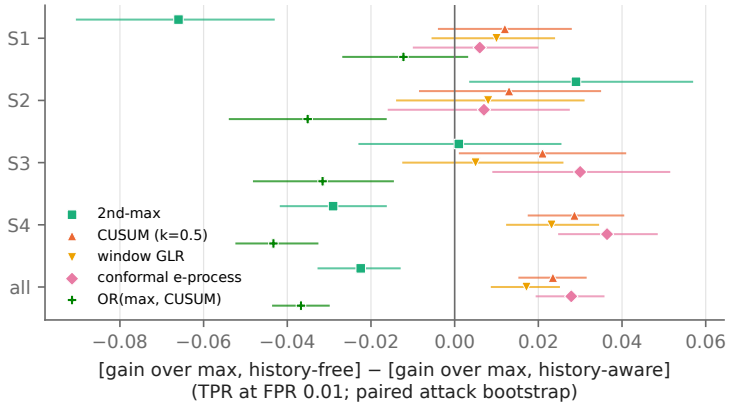


Figure 4: Difference-in-differences: [gain over max, history-free] − [gain over max, history-aware], by structure. The hypothesis predicts positive values for accumulators on S4. They are mostly ≈ 0 or negative.

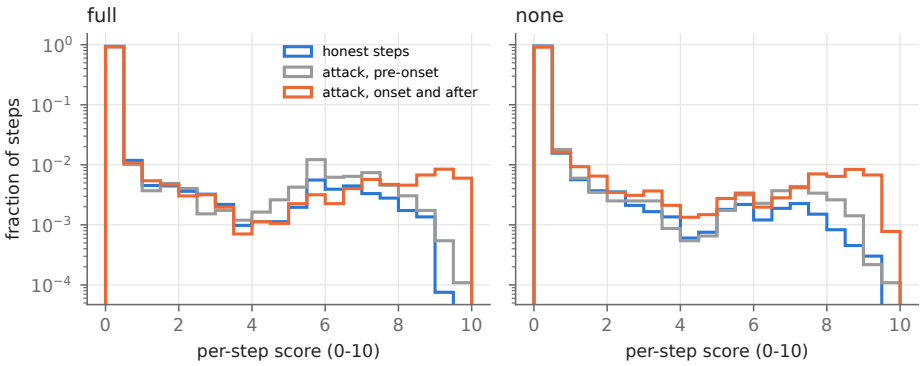


Figure 5: Per-step score distributions (log scale): honest steps, attack steps before onset, and attack steps at or after onset, for both history modes.

4.3 Mechanism across all 46 stream sets

Two properties of a stream set predict whether accumulation beats max (Fig. 6):

- **repeat-spike informativeness:** the log likelihood ratio of a second high step under attack vs honest, given one high step. Positive association: Spearman +0.47 for top-3 sum.
- **honest spike persistence:** P(another high step | one high step) on honest streams. Negative association: −0.67.

This refines the original hypothesis. The relevant variable is not "history-aware vs history-free" as such, but how persistent high scores are in honest vs attack streams. History can move a monitor in either direction: cumulative honest flags (bad for accumulation) or attack re-flagging (good). Both quantities can be estimated from honest data plus a small attack sample *before* choosing a rule.

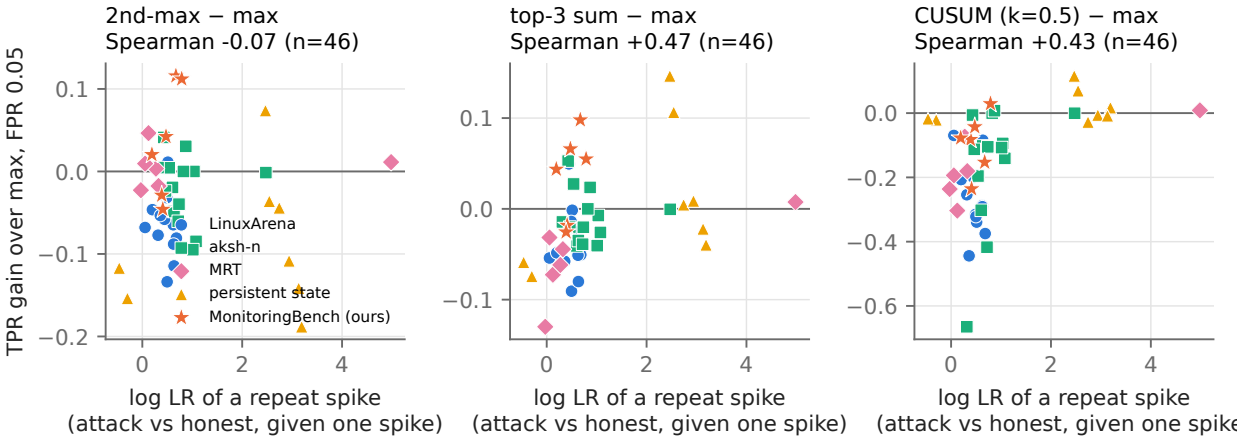


Figure 6: Gain over max vs log likelihood ratio of a repeat spike (attack vs honest), for 2nd-max, top-3 sum and CUSUM.

4.4 Round 3: robustness

Detector ($\alpha = 0.01$)	TPR (full)	TPR (none)
max	0.164	0.146
CUSUM $k = 0.25$	0.024	0.031
CUSUM $k = 0.5$	0.032	0.043
CUSUM $k = 1$	0.097	0.070
CUSUM $k = 1.5$	**0.186**	0.155
CUSUM $k = 2$	0.162	**0.159**
top-2 / top-3 / top-5 sum	0.185 / 0.205 / 0.169	0.198 / 0.186 / 0.149
GLR $W = 5/10/20$	0.144 / 0.083 / 0.058	0.088 / 0.063 / 0.057

Table 4: MonitoringBench parameter grid. Small-k CUSUM and long-window GLR also realize 2–3× the target FPR (0.020–0.031 at $\alpha = 0.01$) on dependent honest streams.

- k **is critical.** On whitened LLM-monitor scores, small k accumulates honest noise. $k \approx 1.5$–2 turns CUSUM into a smoothed max and recovers max-level TPR. The poor showing of $k = 0.5$ CUSUM, SR and long-window GLR in the main tables is partly mis-tuning, not only a failure of accumulation.
- **Dependence breaks i.i.d. guarantees.** Ville ($1/\alpha$) thresholds for e-processes roughly double the FPR on dependent honest streams. Calibrate empirically.
- **Robust to threshold rule.** Conformal (non-randomized) thresholds give the same ranking.
- **Mondrian per-task whitening** was inconclusive: realized FPR was too far above target with per-task honest samples this small (see Idea 2 for per-context thresholds done properly).

5 Conclusions

1. **Keep max, but use 2nd-max / top-3 and length-aware thresholds.** At a matched budget, classic sequential tests with textbook parameters lose to max-type rules on almost every real stream set. 2nd-max and top-3 sum are cheap, robust improvements in several sets. A length-aware null is a large win when honest and attack lengths differ.
2. **Accumulate across sessions with history-free per-session judgments.** This is the one regime with a clean, sizeable gain (+0.15 TPR). To our knowledge it is the first matched-budget evidence that cross-session accumulation of independent per-session scores beats max; the source paper tested only a failing k-strikes rule.
3. **Choose the rule from measurable stream properties** (honest spike persistence; attack repeat-spike informativeness) rather than from whether the monitor sees history.
4. **If using CUSUM on LLM scores, tune** k **for the score distribution** ($k \approx 1.5$–2 on whitened scores here), and calibrate thresholds empirically.

6 Limitations

- **Noisy low-FPR estimates.** Honest sets are small (about 200–550 per set). Logit-ROC extrapolations are in `round1/logit_roc_*.csv` but not used for claims.

- **Heuristic onset labels.** MonitoringBench `attack_step` marks payload execution. The heuristic `first_attack_step` covers 2,055 of 2,442 attacks. The aksh-n ideal-flag step is itself a judgment.
- **One monitor family re-scored** (`gpt-4.1-mini`/`nano`). Window mode was piloted but not scaled.
- **Narrow cross-session evidence.** The cross-session result rests on one PR monitor model (gpt-4o) and 5-PR runs.
- **Static attacks only.** Adaptation to the detector is the subject of Idea 3.

7 Next steps

- Confirm the cross-session result with a second monitor family and longer PR sequences (estimated $10–40).
- Build an adaptive "rule chooser" that estimates honest persistence and repeat-spike informativeness online and picks max, top-k or CUSUM(k).
- Combine with Idea 2's partially pooled per-context baselines.

Reproduction

Code: `agent-monitor-detection/tracks/t1_matched_budget/` and the library `amd/` (detectors, evaluation, scoring). Tables: `results/t1/tables/`; figures: `results/t1/figures/`; per-run CSVs: `results/t1/round{1,2,3}/`; log: `EXPERIMENT_LOG.md`. Repository: https://github.com/gregkocher/agent-monitor-detection (private).